

Part 1 – Deployment & CI/CD Design

Scenario

Spring Boot backend service (sync-service) connected to MongoDB and deployed on GCP VMs.

Environments:

- QA
- Staging
- Production

CI/CD Tool:

- Jenkins

1. Branching Strategy

I would Recommend this Strategy – Git Flow (Simplified)

Branch Structure

Branch	Purpose	Deployment Environment
develop	Active development	QA
staging	Pre-production testing	Staging
main	Stable production code	Production
feature/*	Feature development	No direct deployment
hotfix/*	Emergency production fixes	Production after approval

Workflow

Feature Development

1. Developers create feature branches from develop
2. Code is pushed to feature branch
3. Pull Request (PR) is created
4. Jenkins triggers validation pipeline:

- Build
 - Unit tests
 - Static code analysis
5. After approval, PR is merged into develop
 6. Deployment automatically happens to QA

Staging Deployment

1. Stable code from develop is merged into staging
2. Jenkins deploys application to staging environment
3. QA/UAT testing is performed
4. Performance and smoke testing executed

Production Deployment

1. Only tested code from staging is merged into main
2. Production deployment requires:
 - Manual approval in Jenkins
 - Restricted access for deployment
3. Jenkins deploys to production after approval

Preventing Accidental Production Deployment

The following controls are implemented:

- Production deployment allowed only from main branch
- Manual approval step before production deployment
- RBAC in Jenkins restricting production access
- Branch protection rules enabled in GitHub/GitLab
- Mandatory PR reviews before merge
- Deployment credentials separated by environment

2. Jenkins Pipeline Design

High-Level Pipeline Stages

Git Checkout

↓

Build Application (Maven/Gradle)

↓

Run Unit Tests

↓

Static Code Analysis (SonarQube)

↓

Build Docker Image

↓

Push Image to Artifact Registry

↓

Deploy to Environment

↓

Health Check

↓

Rollback (if deployment fails)

What Happens on PR Merge?

Merge to develop

- Jenkins automatically triggers pipeline
- Application deployed to QA environment
- Smoke tests executed

Merge to staging

- Application deployed to staging
- Integration testing performed

Merge to main

- Manual approval requested
- Production deployment executed
- Monitoring alerts enabled

Rollback Strategy

Recommended Approach – Rolling Deployment with Rollback

If deployment fails:

1. Jenkins detects failed health check
2. Previous stable Docker image tag is identified
3. Previous version is redeployed automatically
4. Alert notification sent to DevOps team

Example:

Current Version: sync-service:v2

Rollback Version: sync-service:v1

Benefits:

- Minimal downtime
- Faster recovery
- Stable rollback process

3. Configuration Management

Environment-Specific Configuration

Separate configuration files are maintained per environment.

Example:

application-qa.yml

application-staging.yml

application-prod.yml

Configurations include:

- MongoDB URLs
- API endpoints
- Logging levels
- Feature flags

Environment selection done using:

SPRING_PROFILES_ACTIVE=qa

SPRING_PROFILES_ACTIVE=staging

SPRING_PROFILES_ACTIVE=prod

Secrets Management

Sensitive data should never be hardcoded.

Secrets include:

- MongoDB credentials
- API keys
- Service account keys

Recommended approach:

- Store secrets in GCP Secret Manager
- Jenkins retrieves secrets securely during deployment
- Access controlled using IAM roles

Example:

MONGO_USERNAME

MONGO_PASSWORD

API_KEY

Benefits:

- Secure secret handling
- Centralized secret management
- Audit logging support

4. Deployment Strategy

Recommended Strategy – Rolling Deployment

Why Rolling Deployment?

Since the application runs on GCP VMs and startup constraints are mentioned, Rolling Deployment is preferred because:

- Lower infrastructure cost
- No need to maintain duplicate environments
- Minimal downtime
- Gradual instance replacement

Why Not Blue-Green?

Blue-Green deployment provides zero downtime but:

- Requires double infrastructure
- Higher cost
- Not ideal for startup budget constraints

Zero/Minimal Downtime Approach

Implementation:

1. Deploy application gradually to instances
2. Health checks verify application readiness
3. Load balancer routes traffic only to healthy instances
4. Old instances removed after successful validation

Additional configurations:

- Readiness probes
- Health check endpoints
- Load balancer integration

Example endpoint:

/actuator/health